

CNN

June 2026

Image

- How to define?

Color

Texture

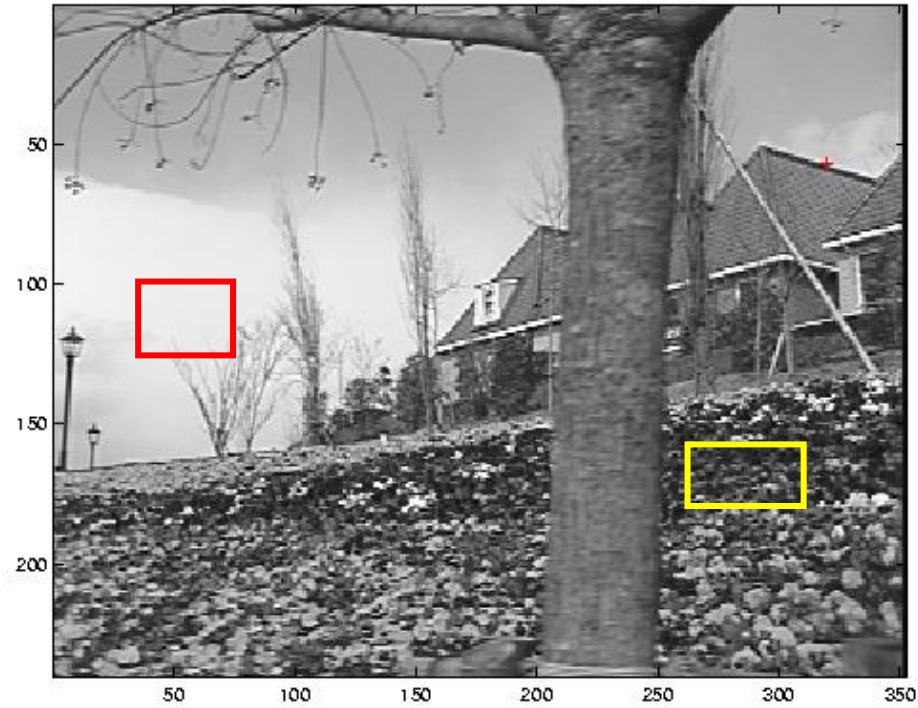
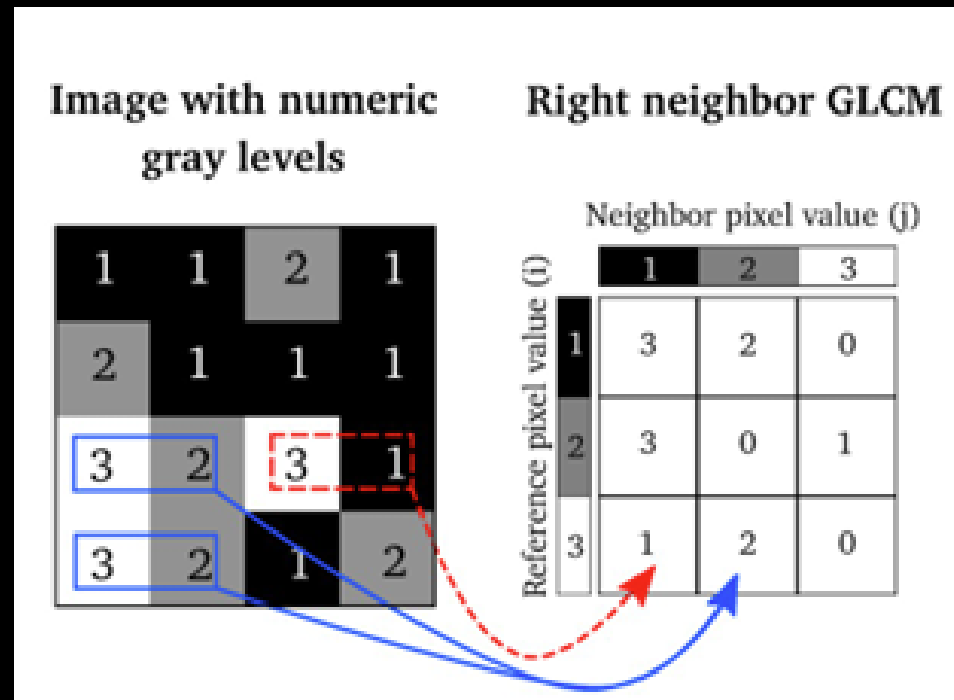
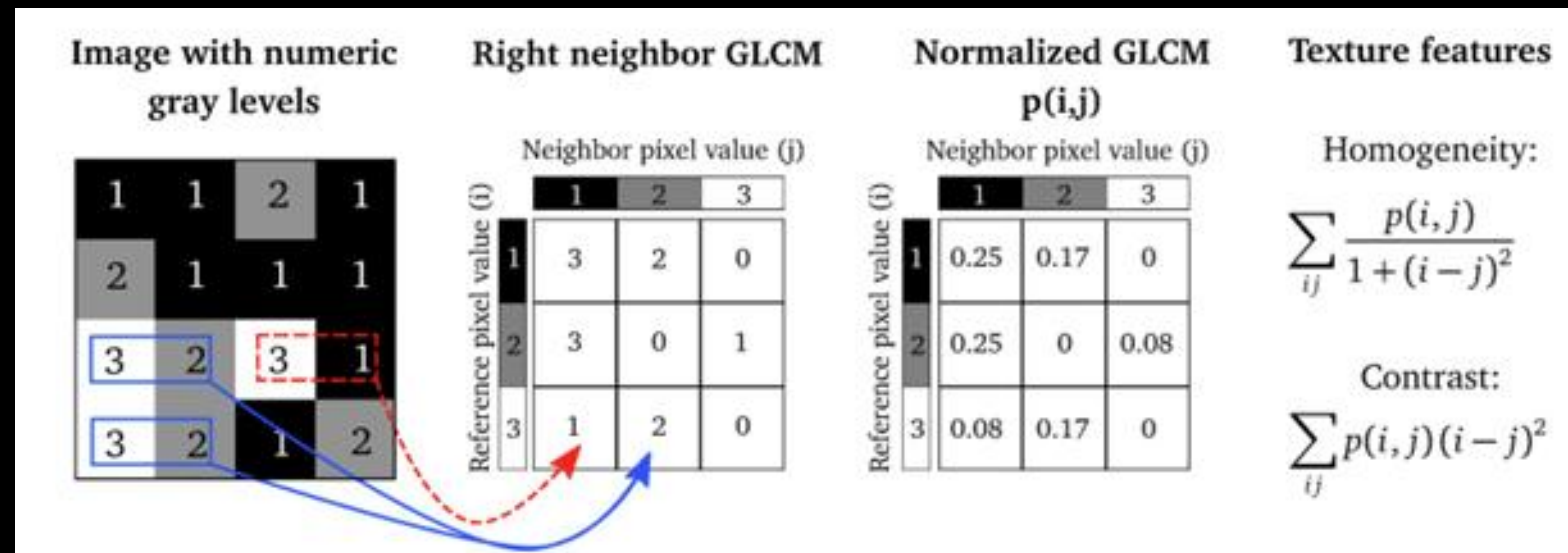


Image partly adopted from Jia-Bin Huang, Virginia Tech

Texture



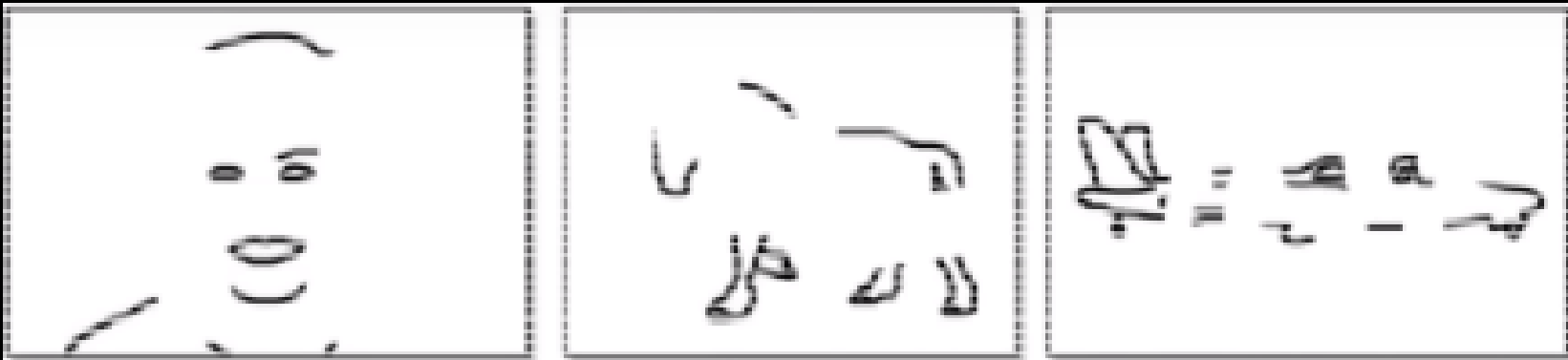
Texture



Edge



Is Edge Useful

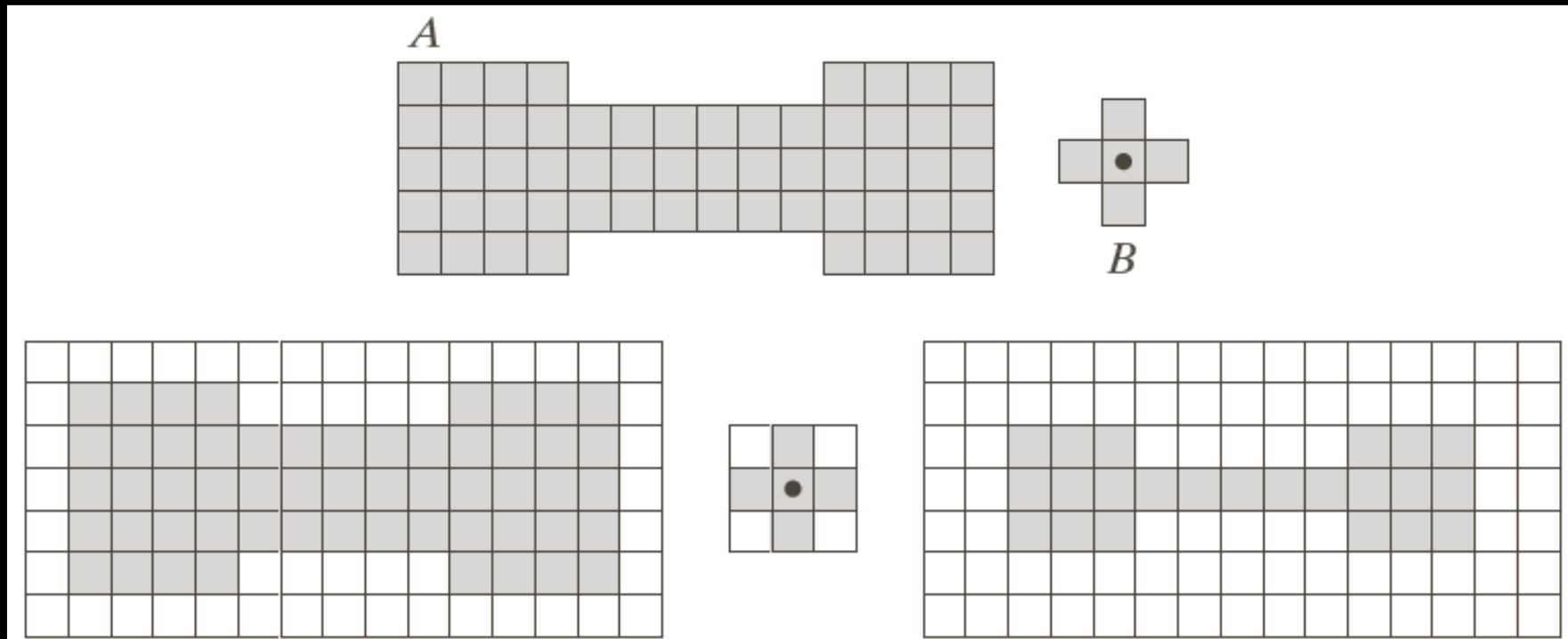


Sobel Operator

-1	0	1
-2	0	2
-1	0	1

1	2	1
0	0	0
-1	-2	-1

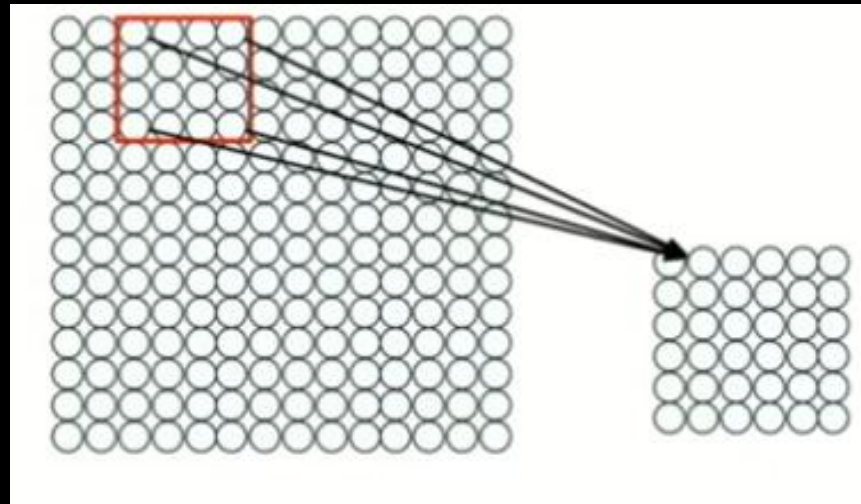
Morphology



Modern Deep Learning

- Modern deep learning – convolutional neural network and other variants
- Convolution is a mathematical way of combining two signals to form a third signal

Convolution



Networks for images

- Problems with fully-connected networks

1. Size

- 224x224 RGB image = 150,528 dimensions
- Hidden layers generally larger than inputs
- One hidden layer = 150,520x150,528 weights -- 22 billion

2. Nearby pixels statistically related

- But could permute pixels and relearn and get same results with FC

3. Should be stable under transformations

- Don't want to re-learn appearance at different parts of image

Convolutional networks

- Parameters only look at local image patches
- Share parameters across image

Convolutional networks

- Networks for images
- Invariance and equivariance
- 1D convolution
- Convolutional layers
- Channels
- Receptive fields
- Convolutional network for MNIST 1D

Invariance

- A function $f[x]$ is **invariant** to a transformation $t[]$ if:

i.e., the function output is the same even after the transformation is applied.

Invariance example

e.g., Image classification

- Image has been translated, but we want our classifier to give the same result



Equivariance

- A function $f[x]$ is **equivariant** to a transformation $t[]$ if:

i.e., the output is transformed in the same way as the input

Equivariance example

e.g., Image segmentation

- Image has been translated and we want segmentation to translate with it



Convolutional networks

- Networks for images
- Invariance and equivariance
- 1D convolution
- Convolutional layers
- Channels
- Receptive fields
- Convolutional network for MNIST 1D

Convolution in 1D

- Input vector $\mathbf{x}$:

$$\mathbf{x} = [x_1, x_2, \dots, x_I]$$

- Output is weighted sum of neighbors:

$$z_i = \omega_1 x_{i-1} + \omega_2 x_i + \omega_3 x_{i+1}$$

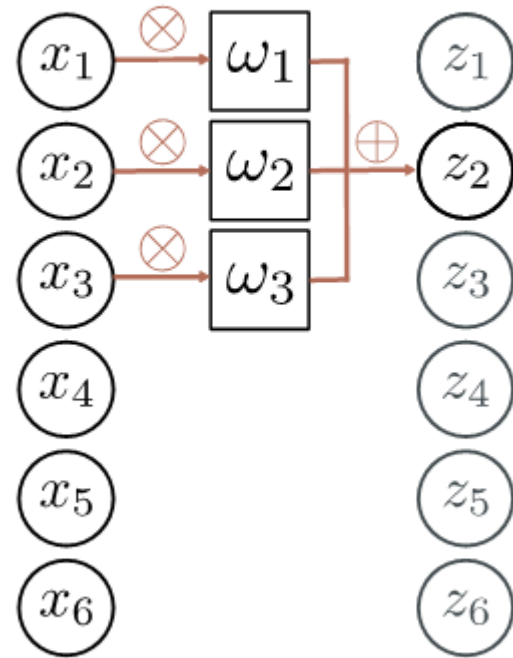
- Convolutional **kernel** or **filter**:

$$\boldsymbol{\omega} = [\omega_1, \omega_2, \omega_3]^T$$

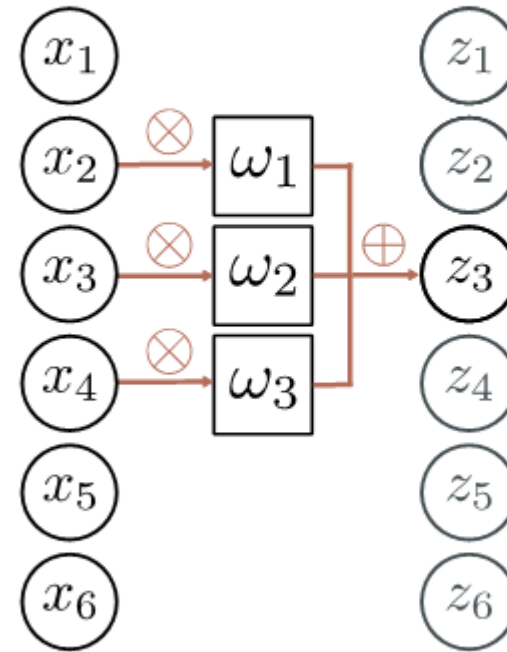

Kernel size = 3

Convolution with kernel size 3

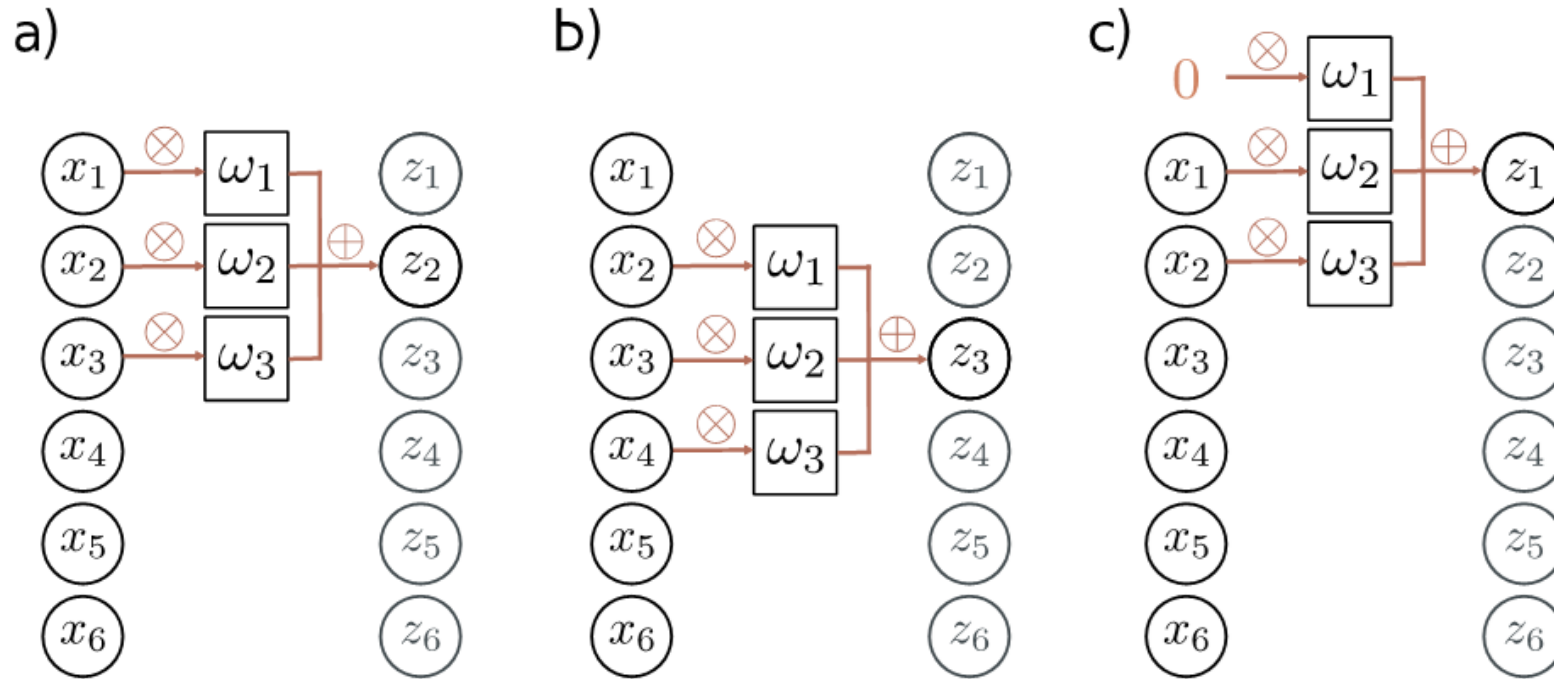
a)



b)



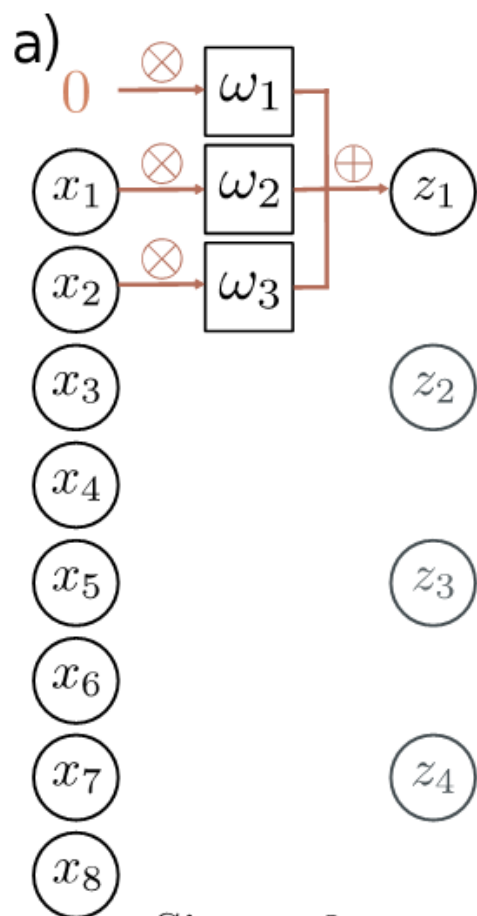
Zero padding



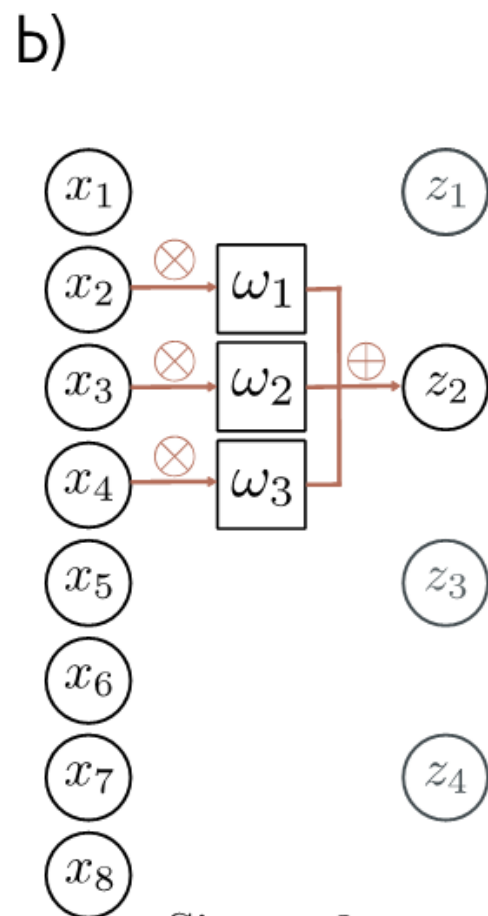
Treat positions that are beyond end of the input as zero.

Stride, kernel size, and dilation

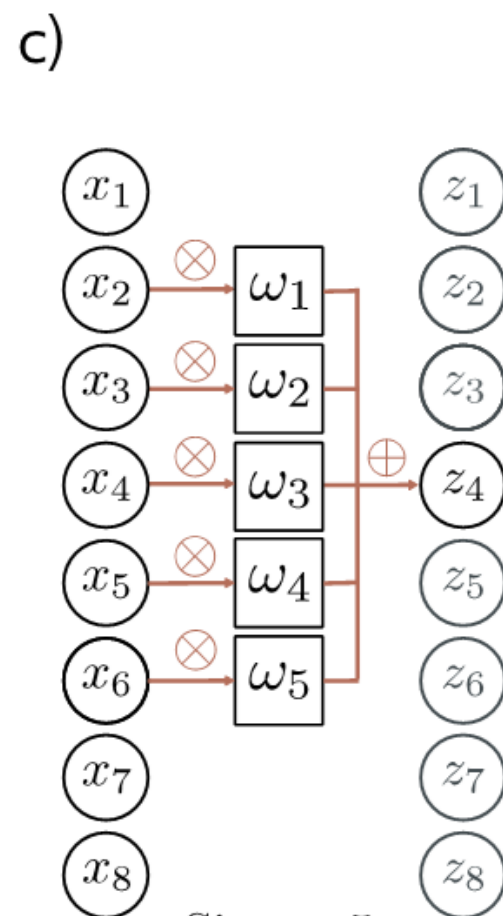
- **Stride** = shift by k positions for each output
 - Decreases size of output relative to input
- **Kernel size** = weight a different number of inputs for each output
 - Combine information from a larger area
 - But kernel size 5 uses 5 parameters
- **Dilated** or **atrous** convolutions = intersperse kernel values with zeros
 - Combine information from a larger area
 - Fewer parameters



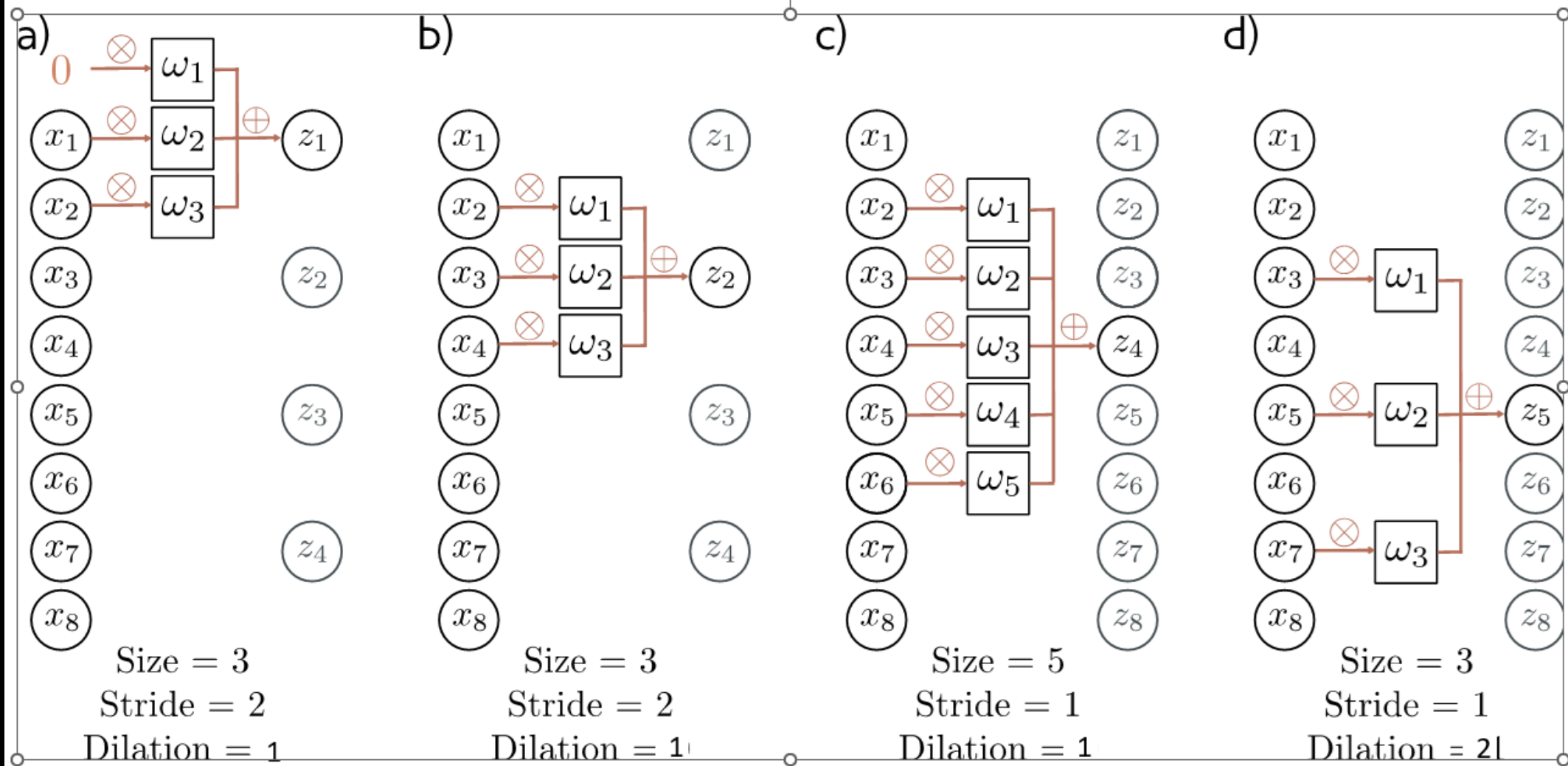
Size = 3
Stride = 2
Dilation = 1



Size = 3
Stride = 2
Dilation = 1



Size = 5
Stride = 1
Dilation = 1



Convolutional layer

$$\begin{aligned} h_i &= a [\beta + \omega_1 x_{i-1} + \omega_2 x_i + \omega_3 x_{i+1}] \\ &= a \left[\beta + \sum_{j=1}^3 \omega_j x_{i+j-2} \right] \end{aligned}$$

Convolutional layer

Convolutional network:

$$\begin{aligned} h_i &= a [\beta + \omega_1 x_{i-1} + \omega_2 x_i + \omega_3 x_{i+1}] \\ &= a \left[\beta + \sum_{j=1}^3 \omega_j x_{i+j-2} \right] \end{aligned}$$

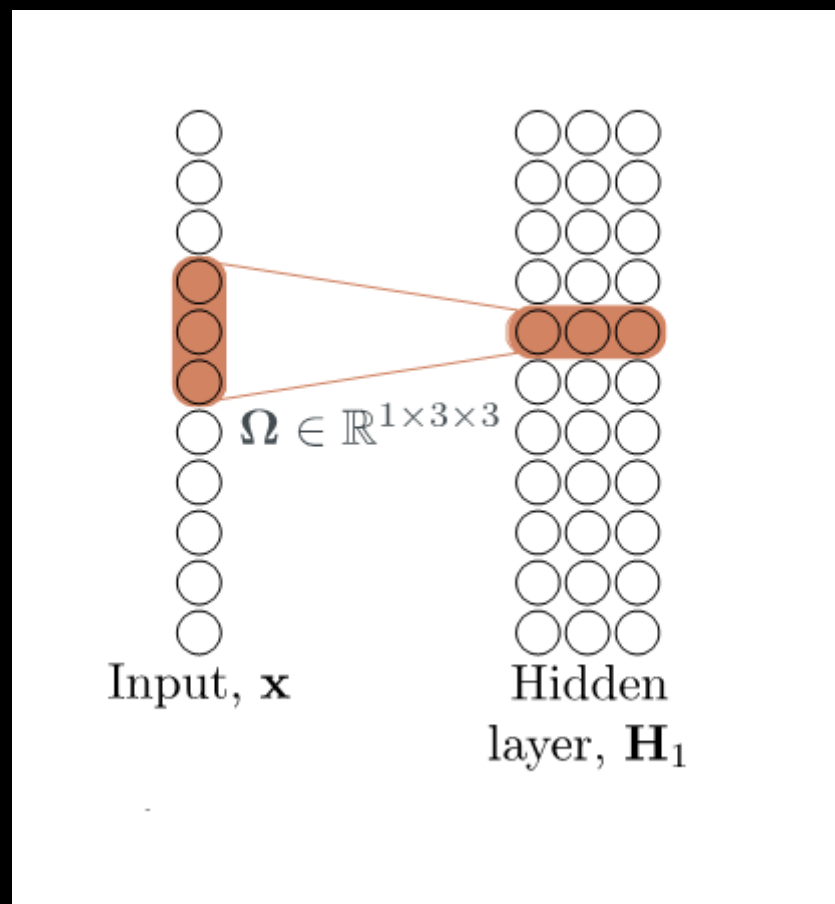

3 weights, 1 bias

Fully connected network:

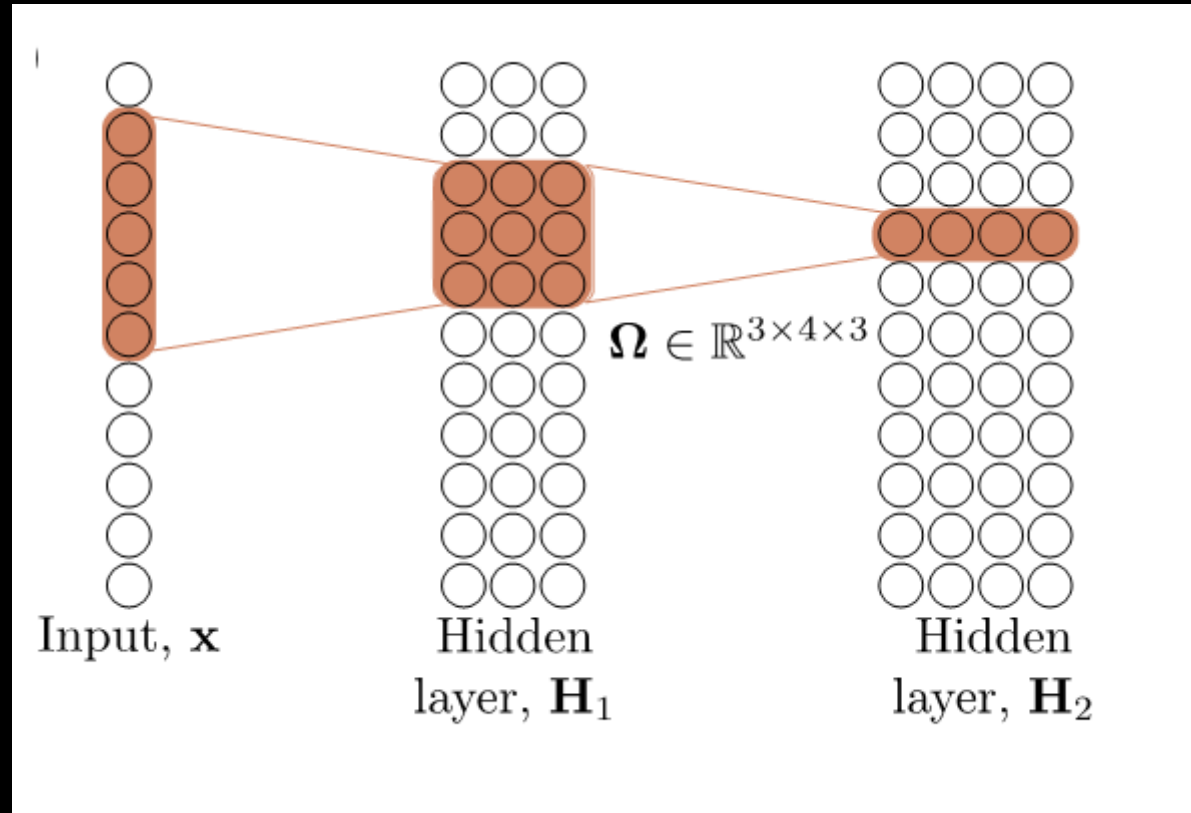
$$h_i = a \left[\beta_i + \sum_{j=1}^D \omega_{ij} x_j \right]$$


D^2 weights, D biases

Receptive Fields

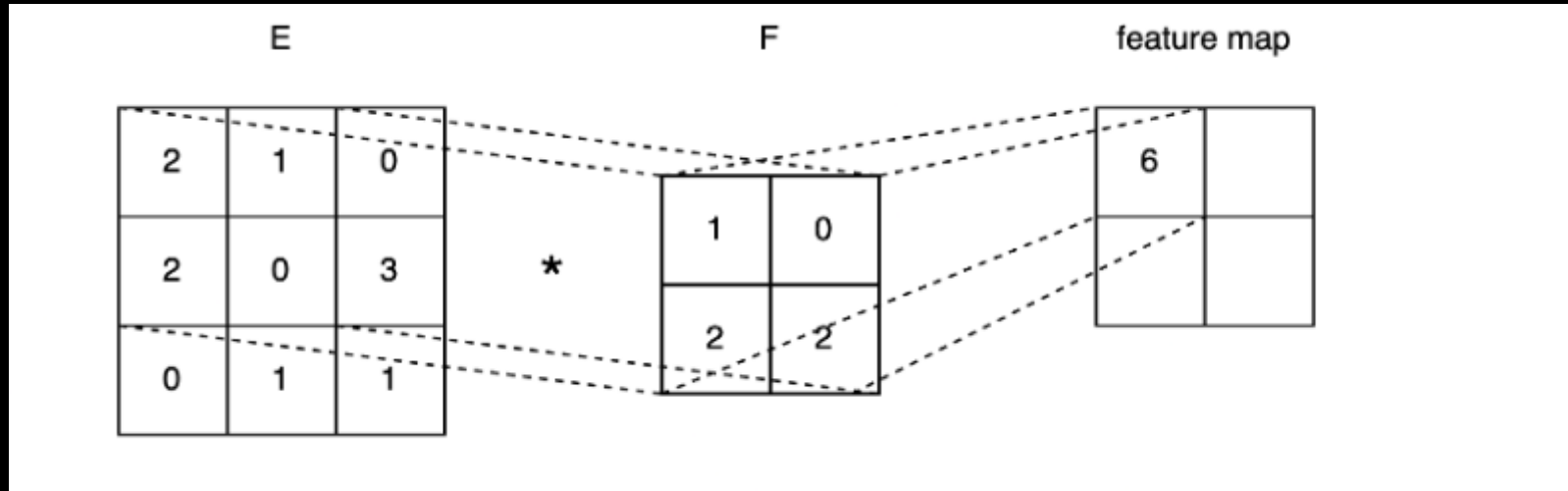


Receptive Fields

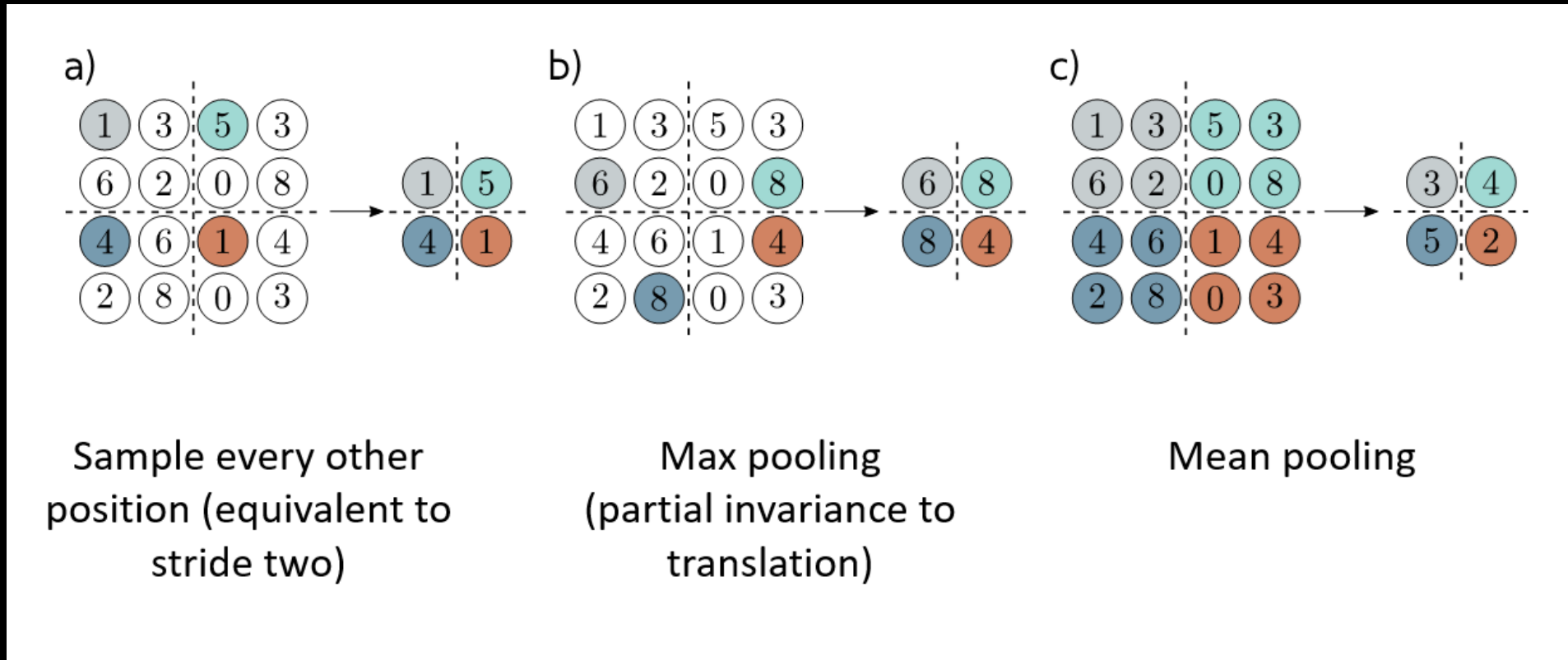


Weight initialization

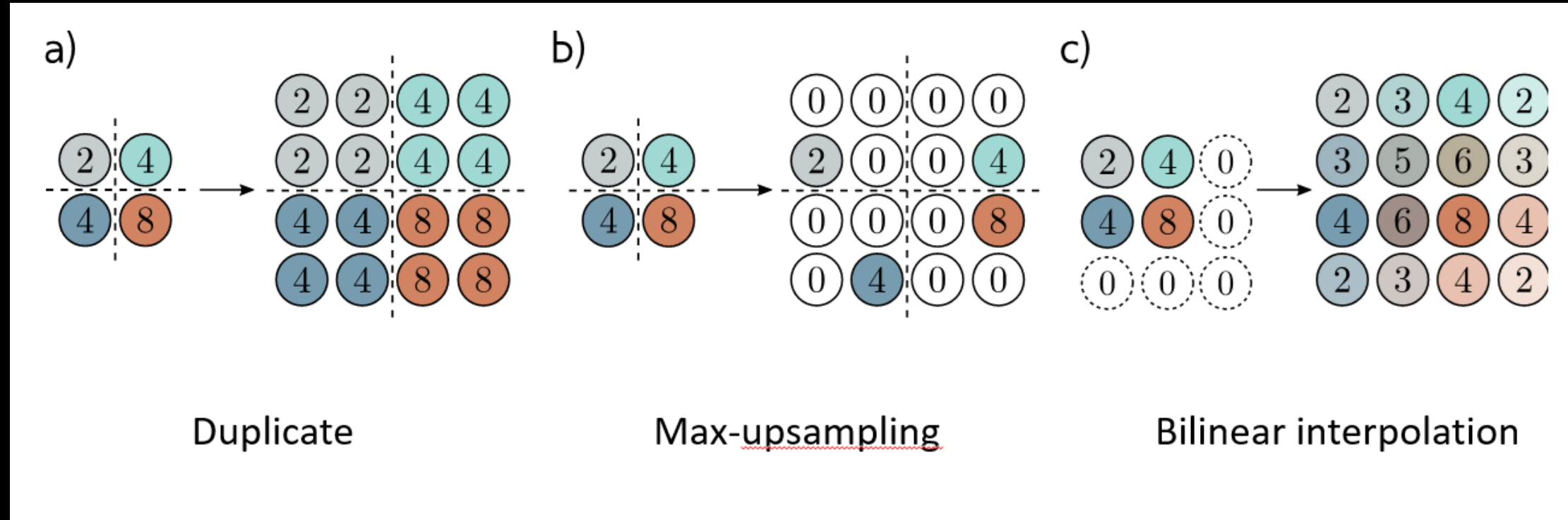
2D CNN



Downsampling



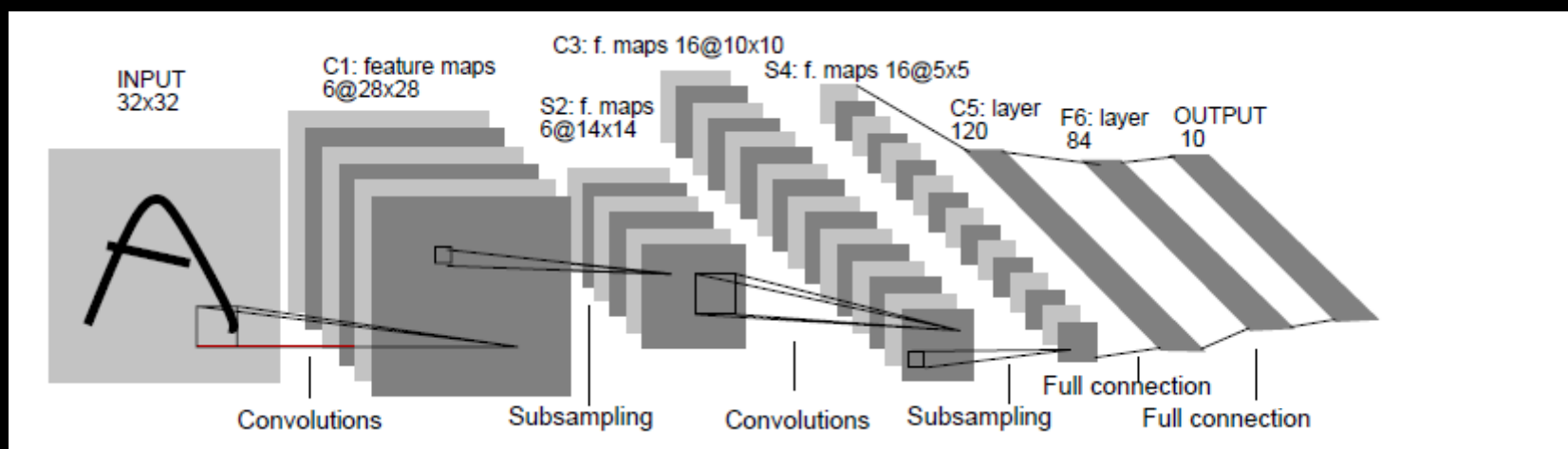
Upsampling



Upsampling

- Mostly done using transposed convolution

LeNet



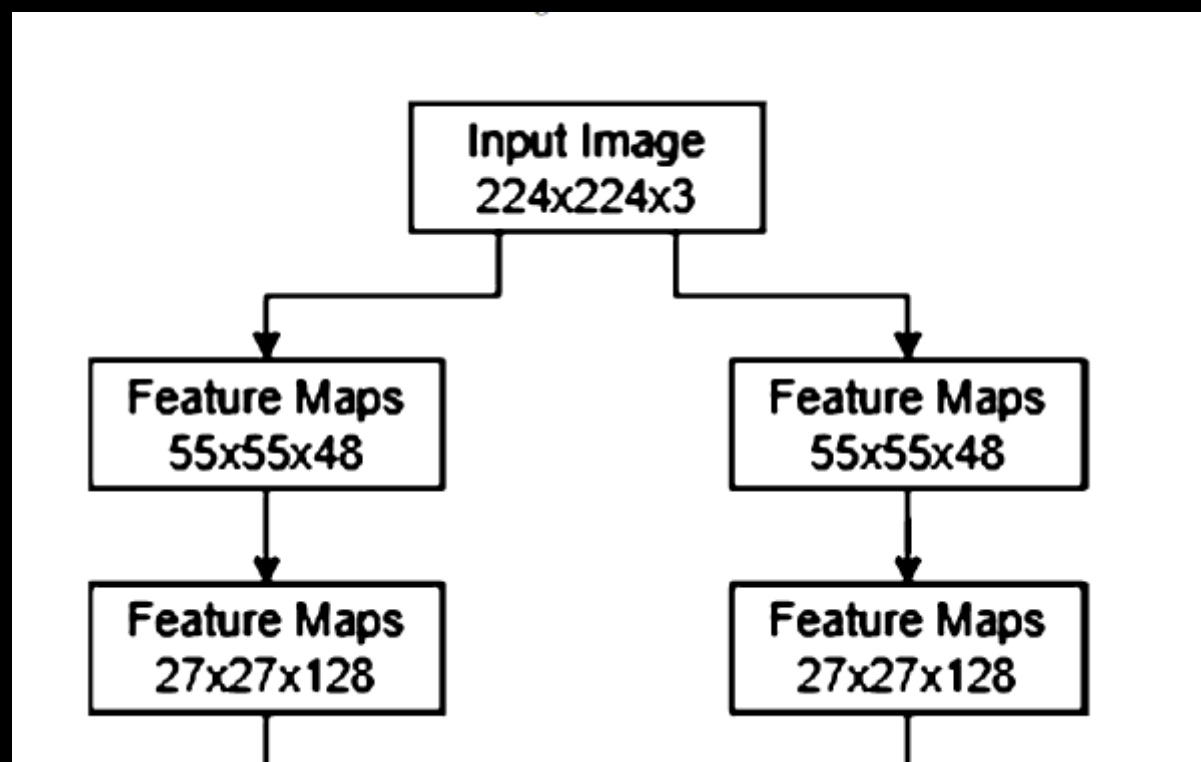
ImageNet Large Scale Visual Recognition Challenge 2010 (ILSVRC2010)

- 1000 object categories

- Descriptor Coding + SVM, 0.28 --- NEC-UIUC
- Fisher kernel + SVM, 0.34 --- XRCE
- LI2C, 0.58 --- NTU_WZX

https://image-net.org/static_files/files/pascal_ilsvrc.pdf

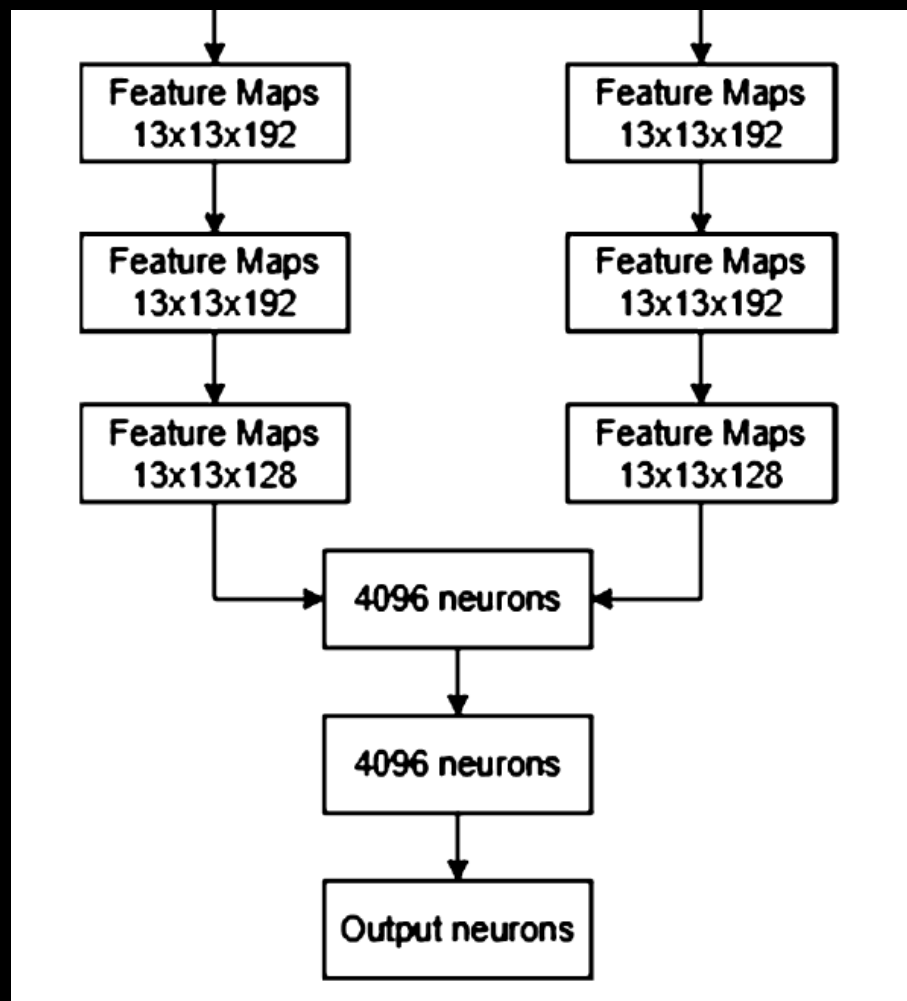
AlexNet



AlexNet

The architecture of AlexNet is divided into two distinct parts, a design choice influenced by the limited memory capacity of GPUs. As a result, the network's architecture was split across two GPUs, with each GPU managing half of the neurons or feature maps in specific layers.

AlexNet



AlexNet

Five convolutional and three fully connected layers

The first convolutional layer uses 96 kernels of spatial size 11×11 . These large filters allowed the network to capture complex and high-level features from images.

The subsequent convolutional layers use kernels of size 3×3 .

AlexNet further used

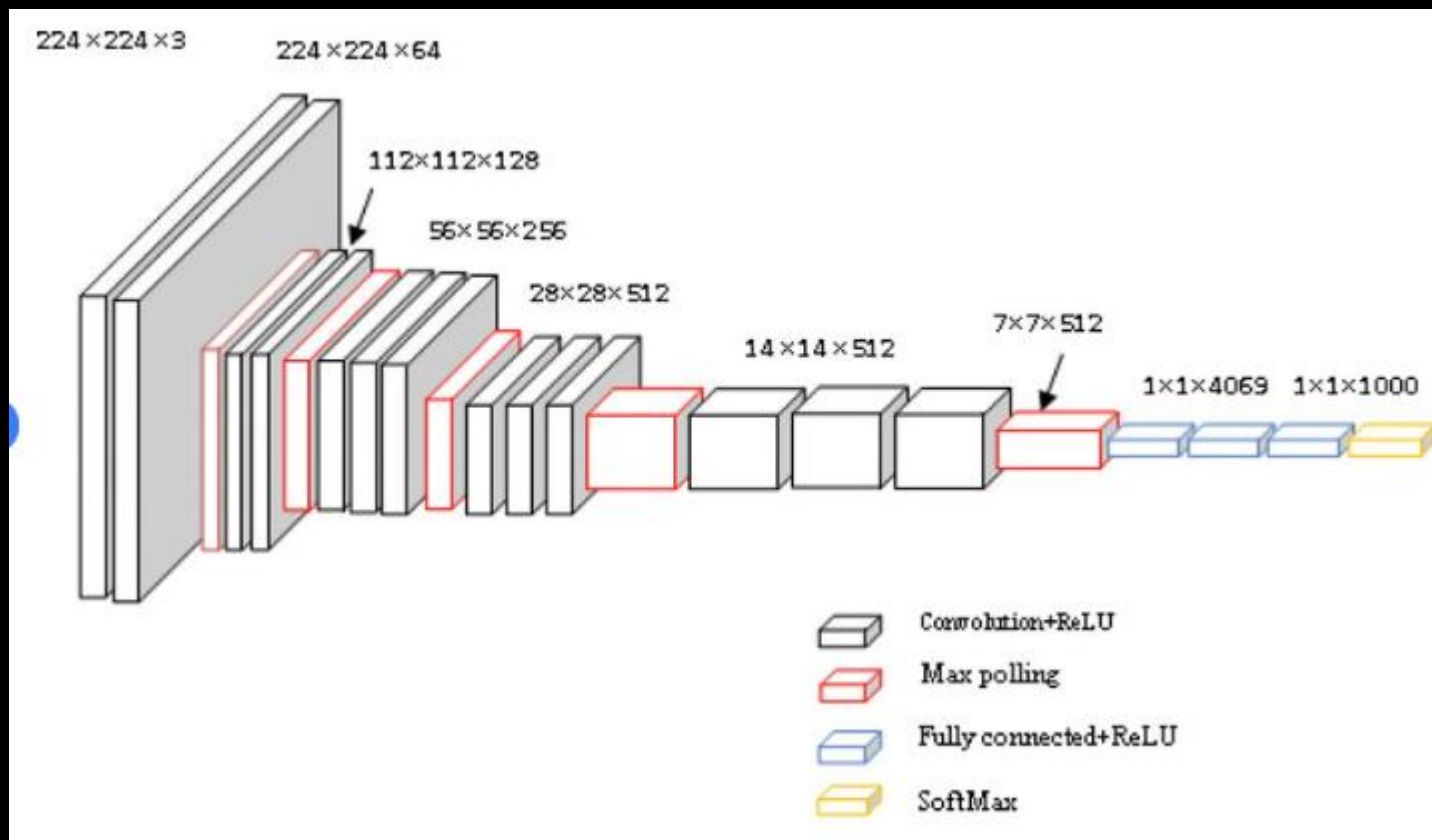
- ✓ DropOut
- ✓ ReLU
- ✓ Max Pooling

- AlexNet advocated for extensive data augmentation as a way to reduce overfitting, a practice that has since then become a norm.

VGGNet

- Unlike AlexNet, VGGNet promoted the use of 3×3 convolution filters. It suggested that maintaining a consistent convolution filter size across all layers and simply increasing the depth of the CNN could improve performance on visual recognition tasks.

VGGNet



https://www.researchgate.net/figure/GG-Neural-Network-Architecture-Source_fig5_369178183

VGGNet

Their study suggests that, although VGGNet has more layers and a larger number of parameters compared to previous models, it requires fewer epochs for training. This can be partially attributed to the effect of smaller filter sizes, while pre-initialized weights also playing a significant role.

Does Deeper Network Perform Better?

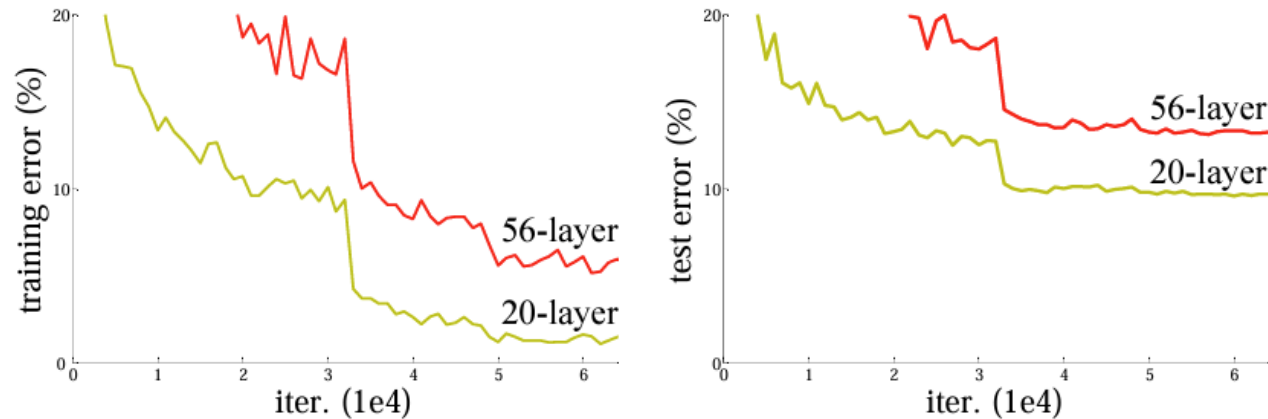
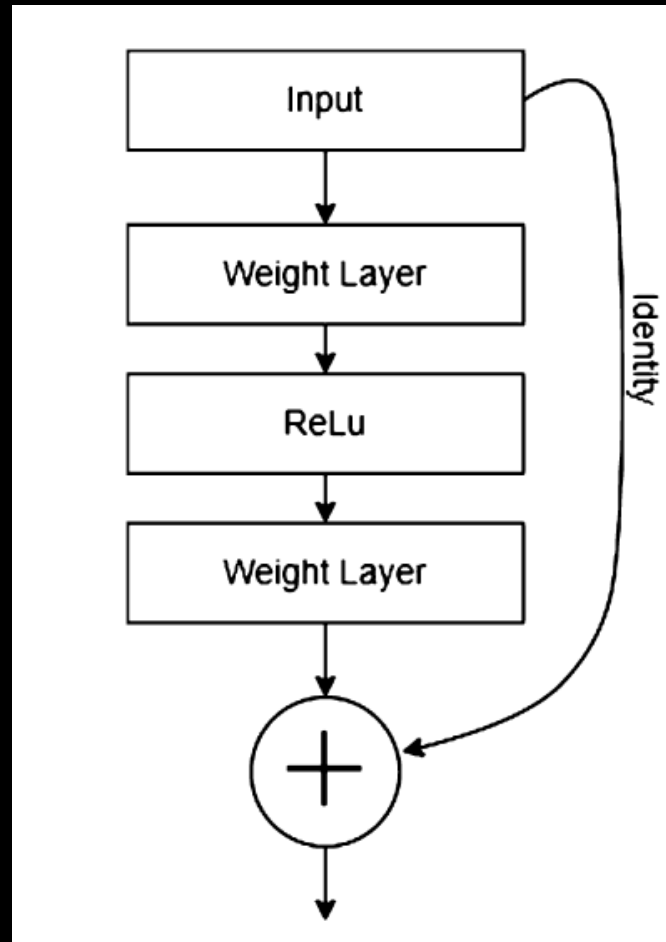


Figure 1. Training error (left) and test error (right) on CIFAR-10 with 20-layer and 56-layer “plain” networks. The deeper network has higher training error, and thus test error. Similar phenomena on ImageNet is presented in Fig. 4.

Residual Block




```
def forward(self, x: Tensor) -> Tensor:
    identity = x

    out = self.conv1(x)
    out = self.bn1(out)
    out = self.relu(out)

    out = self.conv2(out)
    out = self.bn2(out)

    out += identity
    out = self.relu(out)

    return out
```

	plain	ResNet
18 layers	27.94	27.88
34 layers	28.54	25.03

Table 2. Top-1 error (% , 10-crop testing) on ImageNet validation.

Deep Residual Learning for Image Recognition, 2015